

week 2: simulation results & make your own algorithm simulation

```
import numpy as np
import pandas as pd
pd.options.mode.chained_assignment = None
import pickle

import os
from importlib import reload

import matplotlib.pyplot as plt
import matplotlib
# import seaborn as sns
%matplotlib inline
plt.style.use('ggplot')
from matplotlib.ticker import MultipleLocator
```

part 1: load competition details & visualize

```
df_comp_details = pd.read_csv('duopoly_competition_details.csv')
df_comp_details.fillna(0, inplace=True)
df_comp_details.head(3)
```

	competition_id	selling_season	selling_period	competitor_id \
0	3HmCSH	1	1	SaffronDolphin
1	3HmCSH	1	2	SaffronDolphin
2	3HmCSH	1	3	SaffronDolphin

	price_competitor	price	demand	competitor_has_capacity \
0	56.1	66.4	0	True
1	66.4	66.4	0	True
2	66.4	66.4	0	True

	calculation_duration	errors
0	0.005	0.0
1	0.006	0.0
2	0.007	0.0

check how many competitions (one set it 100 selling seasons) we ran against each competitor

- when two opponents are selected for a 'battle' they perform 4 competitions
- during one night you can be drawn twice to compete against the same competitor

```
df_comp_details.groupby('competitor_id').agg({  
    'competition_id' : 'nunique',  
    'selling_season' : ['max', 'count']  
})
```

	competition_id	selling_season	
	nunique	max	count
competitor_id			
AttentiveOriole	2	100	20000
SaffronDolphin	2	100	20000

```
df_comp_details['unique_selling_season_key'] =  
df_comp_details.apply(lambda r:  
    "%s_%s" %  
(r.competition_id,r.selling_season), axis=1)
```

```
df_comp_details.head(3)
```

	competition_id	selling_season	selling_period	competitor_id \
0	3HmCSH	1	1	SaffronDolphin
1	3HmCSH	1	2	SaffronDolphin
2	3HmCSH	1	3	SaffronDolphin

	price_competitor	price	demand	competitor_has_capacity \
0	56.1	66.4	0	True
1	66.4	66.4	0	True
2	66.4	66.4	0	True

	calculation_duration	errors	unique_selling_season_key
0	0.005	0.0	3HmCSH_1
1	0.006	0.0	3HmCSH_1
2	0.007	0.0	3HmCSH_1

part 2: simulate a basic pricing algorithm with data from the competition details CSV

When simulating the DPC demand algorithms based on the competition details CSV, consider the shifted time information structure

- when requested to return the price for selling period 1, we only have the information that the competitor has capacity to sell, but no demand and pricing information from the previous selling period (since there is no)
- when requesting the price for selling_period 2, the pricing function will receive the information on the prices (including the competitor's) from the last period (1) and the realized demand from the previous selling_period
- ...

competition_id	selling_season	selling_period	competitor_id	price_competitor	price	demand	competitor_has_capacity
YCGzvi	1	1	RussetBat	50	80	0	TRUE
YCGzvi	1	2	RussetBat	52	78.9	0	TRUE
YCGzvi	1	3	RussetBat	54	77.3	1	TRUE
YCGzvi	1	4	RussetBat	56	75.3	2	TRUE
YCGzvi	1	5	RussetBat	56	72.8	0	TRUE
YCGzvi	1	6	RussetBat	58	69.8	2	TRUE

pre-step: delete potential existing 'duopoly_feedback.data'

```
if os.path.isfile('duopoly_feedback.data'):
    os.remove('duopoly_feedback.data')
    print('duopoly_feedback.data deleted')
else:
    print('duopoly_feedback.data does not exists')
```

duopoly_feedback.data does not exists

reload the pricing algo, in case you made changes

```
import duopoly_simple as duopoly
reload(duopoly)

<module 'duopoly_simple' from '/mnt/c/Users/Alwin/Haensel AMS
Dropbox/Alwin
Haensel/IMPORTANT/RESEARCH_TEACHING/Beuth_Teaching/Optimization_Learnin
g_WS2025/W_02_Alwin/simulation/duopoly_simple.py'>

df_comp_details.demand.describe()
```

```

count      40000.000000
mean       0.780400
std        0.958175
min        0.000000
25%        0.000000
50%        1.000000
75%        1.000000
max        7.000000
Name: demand, dtype: float64

```

loop over the selling_period (starting at 1 with the initialization) and call the price function from your algo

```

s = 1 # first selling season
max_t = 20 # selling period until we want to run the simulation
information_dump = None
my_prices = []

# select form the competition details CSV which selling_season
# you want to use for the simulation
id = df_comp_details['unique_selling_season_key'].unique()[0]
df_select =
df_comp_details[df_comp_details.unique_selling_season_key==id]

# loop over the selling periods t
for t in np.arange(1,(max_t+1),1):

    print("we are in selling_period = %d" % t)

    if t <=1 :
        prices_historical = None
        demand_historical = None
    else:
        # get price info from details CSV
        prices_historical = [ my_prices,
                             df_select[df_select.selling_period < t]
                             ['price_competitor'] ]
        prices_historical = np.array(prices_historical)
        # get demand info from details CSV
        demand_historical =
np.array(df_select[df_select.selling_period < t]['demand'])

        # get current info if competitor can sell
        competitor_has_capacity = df_select[df_select.selling_period == t]
        ['competitor_has_capacity'].values[0]

        # build input object for price algo
        request_input = {
            "current_selling_season" : s,

```

```

        "selling_period_in_current_season" : t,
        "prices_historical_in_current_season" : prices_historical,
        "demand_historical_in_current_season" : demand_historical,
        "competitor_has_capacity_current_period_in_current_season" :
competitor_has_capacity,
        "information_dump": information_dump
    }
    # request price from you algo and receive (price,
information_dump)
    price , information_dump = duopoly.p(**request_input)

    print("###")
    print("We received from the algo the following results:")
    print("price: %.2f" % price)
    print("information_dump: %s" % str(information_dump) )
    # add you price to the own prices array
    my_prices.append(price)

we are in selling_period = 1
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 1
    Algo: setting random price at init: 35.570
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 35.57
    Algo: ---
###
We received from the algo the following results:
price: 35.57
information_dump: {'revenue': 0}
we are in selling_period = 2
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 2
    Algo: random_draw_4_price_change: 0.707 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 35.57
    Algo: ---
###
We received from the algo the following results:
price: 35.57
information_dump: {'revenue': 0.0}
we are in selling_period = 3
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 3
    Algo: random_draw_4_price_change: 0.183 => setting new price:
True
    Algo: new price is: 69.39
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 69.39

```

```

    Algo: ---
###
We received from the algo the following results:
price: 69.39
information_dump: {'revenue': 0.0}
we are in selling_period = 4
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 4
    Algo: random_draw_4_price_change: 0.979 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 69.39
    Algo: ---
###
We received from the algo the following results:
price: 69.39
information_dump: {'revenue': 0.0}
we are in selling_period = 5
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 5
    Algo: random_draw_4_price_change: 0.035 => setting new price:
True
    Algo: new price is: 60.58
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 60.58
    Algo: ---
###
We received from the algo the following results:
price: 60.58
information_dump: {'revenue': 0.0}
we are in selling_period = 6
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 6
    Algo: random_draw_4_price_change: 0.619 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 60.58
    Algo: ---
###
We received from the algo the following results:
price: 60.58
information_dump: {'revenue': 0.0}
we are in selling_period = 7
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 7
    Algo: random_draw_4_price_change: 0.724 => setting new price:
False

```

```

    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 60.58
    Algo: ---
###
We received from the algo the following results:
price: 60.58
information_dump: {'revenue': 0.0}
we are in selling_period = 8
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 8
    Algo: random_draw_4_price_change: 0.170 => setting new price:
True
    Algo: new price is: 58.37
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 58.37
    Algo: ---
###
We received from the algo the following results:
price: 58.37
information_dump: {'revenue': 0.0}
we are in selling_period = 9
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 9
    Algo: random_draw_4_price_change: 0.635 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 58.37
    Algo: ---
###
We received from the algo the following results:
price: 58.37
information_dump: {'revenue': 0.0}
we are in selling_period = 10
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 10
    Algo: random_draw_4_price_change: 0.607 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 58.37
    Algo: ---
###
We received from the algo the following results:
price: 58.37
information_dump: {'revenue': 0.0}
we are in selling_period = 11
    Algo: ---

```

```

    Algo: we are in selling_season 1 and selling_period 11
    Algo: random_draw_4_price_change: 0.663 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 0.00
    Algo: my next price is: 58.37
    Algo: ---
###
We received from the algo the following results:
price: 58.37
information_dump: {'revenue': 0.0}
we are in selling_period = 12
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 12
    Algo: random_draw_4_price_change: 0.054 => setting new price:
True
    Algo: new price is: 41.75
    Algo: so far we have generated revenue: 58.37
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75
information_dump: {'revenue': 58.365821148080016}
we are in selling_period = 13
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 13
    Algo: random_draw_4_price_change: 0.440 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 100.12
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75
information_dump: {'revenue': 100.11714400199553}
we are in selling_period = 14
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 14
    Algo: random_draw_4_price_change: 0.436 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 100.12
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75

```



```
information_dump: {'revenue': 100.11714400199553}
we are in selling_period = 15
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 15
    Algo: random_draw_4_price_change: 0.592 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 100.12
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75
information_dump: {'revenue': 100.11714400199553}
we are in selling_period = 16
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 16
    Algo: random_draw_4_price_change: 0.652 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 141.87
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75
information_dump: {'revenue': 141.86846685591104}
we are in selling_period = 17
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 17
    Algo: random_draw_4_price_change: 0.670 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 225.37
    Algo: my next price is: 41.75
    Algo: ---
###
We received from the algo the following results:
price: 41.75
information_dump: {'revenue': 225.37111256374206}
we are in selling_period = 18
    Algo: ---
    Algo: we are in selling_season 1 and selling_period 18
    Algo: random_draw_4_price_change: 0.354 => setting new price:
False
    Algo: keeping old price
    Algo: so far we have generated revenue: 225.37
    Algo: my next price is: 41.75
    Algo: ---
```

###

We received from the algo the following results:

price: 41.75

information_dump: {'revenue': 225.37111256374206}

we are in selling_period = 19

Algo: ---

Algo: we are in selling_season 1 and selling_period 19

Algo: random_draw_4_price_change: 0.593 => setting new price:

False

Algo: keeping old price

Algo: so far we have generated revenue: 308.87

Algo: my next price is: 41.75

Algo: ---

###

We received from the algo the following results:

price: 41.75

information_dump: {'revenue': 308.8737582715731}

we are in selling_period = 20

Algo: ---

Algo: we are in selling_season 1 and selling_period 20

Algo: random_draw_4_price_change: 0.484 => setting new price:

False

Algo: keeping old price

Algo: so far we have generated revenue: 350.63

Algo: my next price is: 41.75

Algo: ---

###

We received from the algo the following results:

price: 41.75

information_dump: {'revenue': 350.6250811254886}